LAB 3: ADC, DAC and DMA

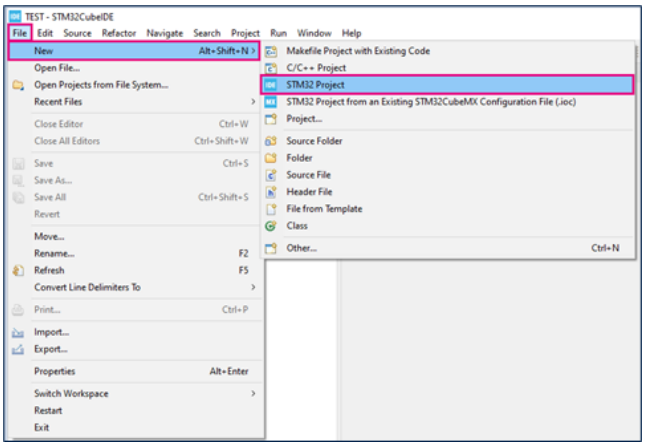
Part 1: Configure ADC to measure the DAC output

1.1. Objectives

- Learn How to use the LCD 1602 in STM32L476
- Learn how to set up the UART with DMA in STM32CubeMX.
- Generate code in STM32CubeMX and using HAL functions.
- Create a simple application to start the ADC and measure the DAC output.

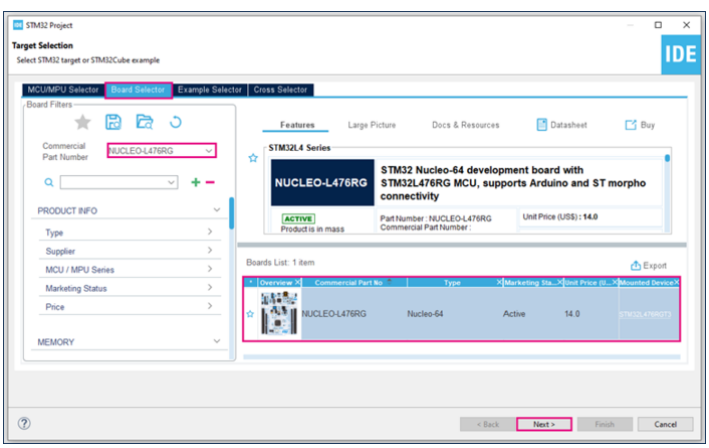
1.2. Create the project in STM32CubeIDE

- **File** > **New** > **STM32 Project** in main panel.



This example uses the NUCLEO-L476RG board.

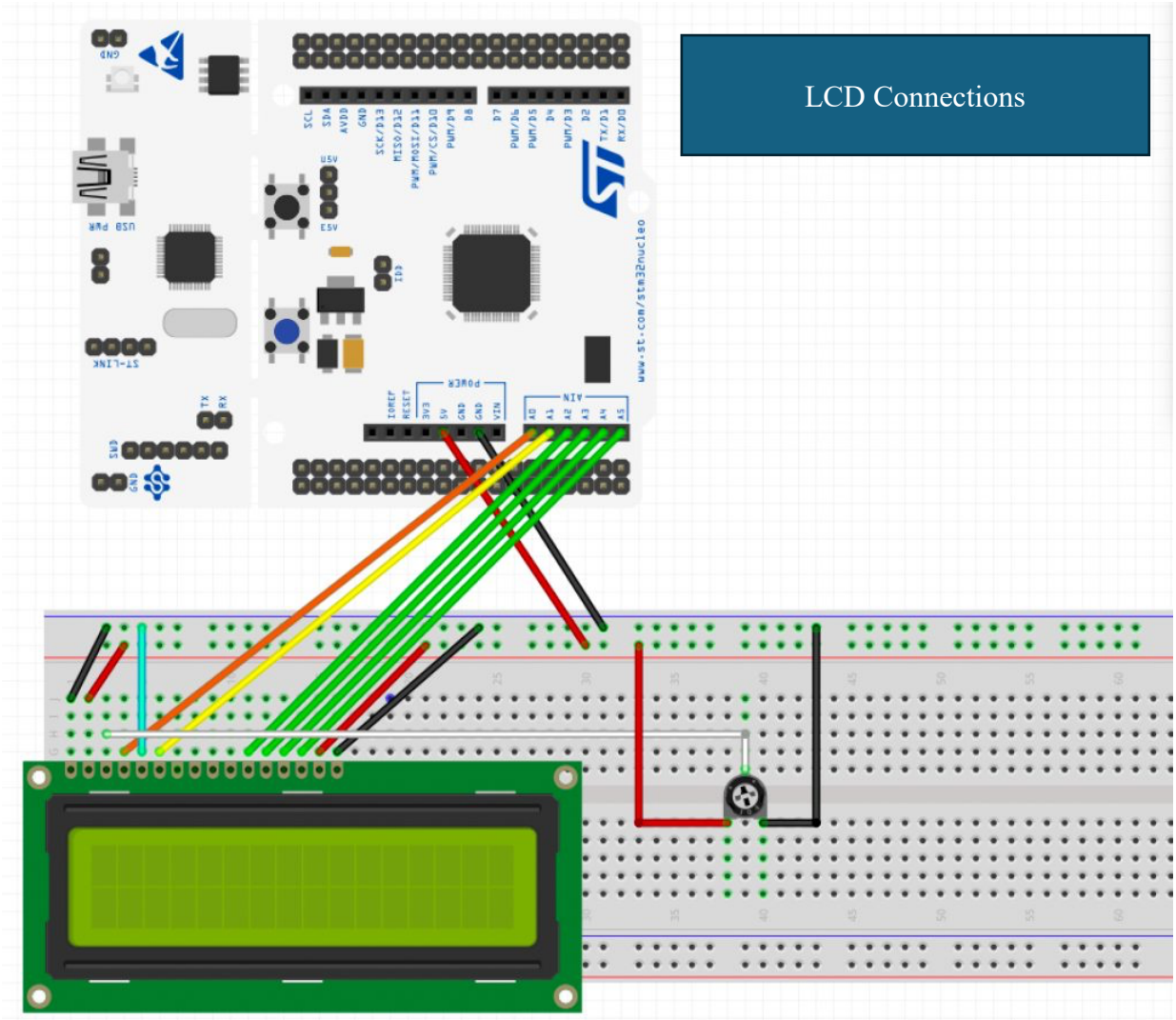
- Select **NUCLEO-L476RG** board using **Board selector** as shown in the figure below:



- Save the project:

1.3. Configure peripherals

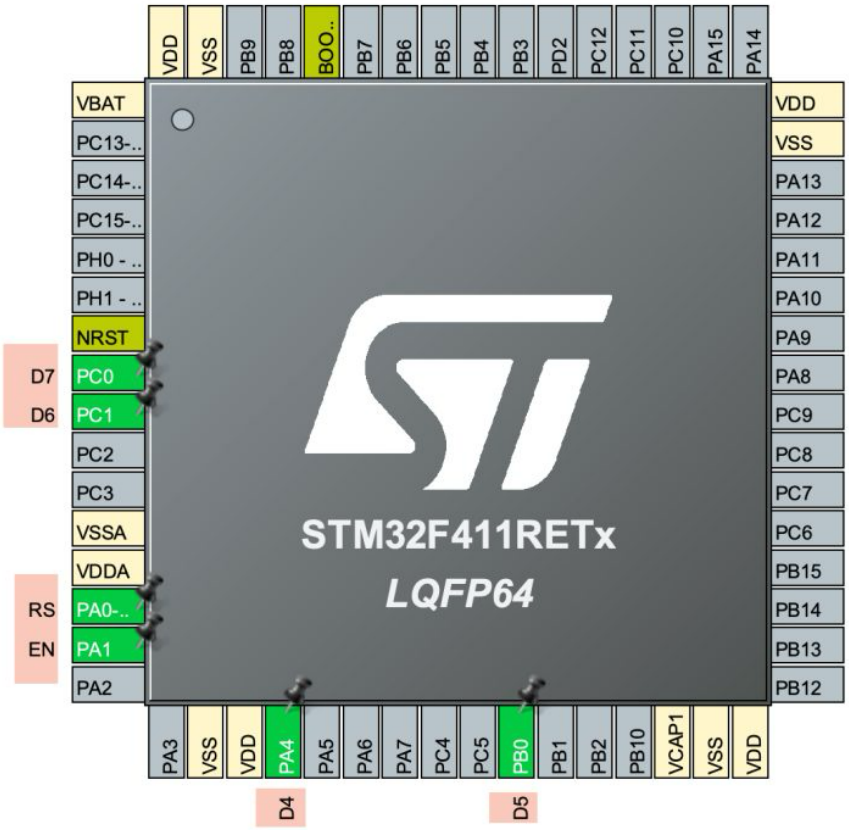
1.3.1. Configure LCD



The pins of LCD1602 as following:

LCD1602 Display	STM32F476 Nucleo-64
Vss	GND
Vdd	5V
Ve	Middle of Potentiometer
Register Select	PA0 (A0 of Arduino Uno Pin)
Read/Write	GND
Enable	PA1 (A1 of Arduino Uno Pin)
D0-D3	Not connected
D4	PA4 (A2 of Arduino Uno Pin)
D5	PB0 (A3 of Arduino Uno Pin)
D6	PC1 (A4 of Arduino Uno Pin)
D7	PC0 (A5 of Arduino Uno Pin)
LED+	5V
LED-	GND

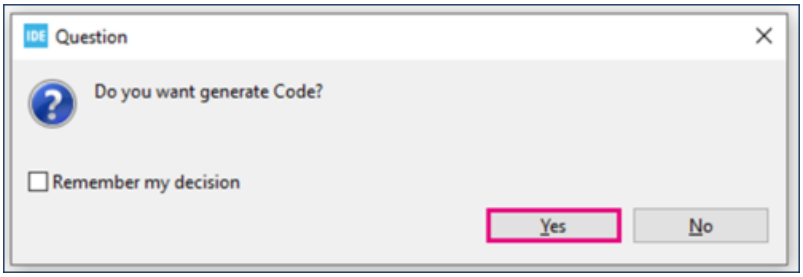
Connected Pins



Give each GPIO a name as shown in the figure above.

Generate the project

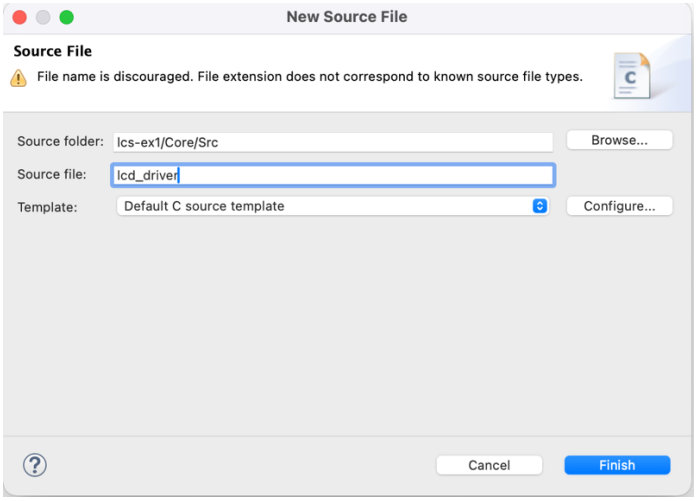
- Click **Ctrl+S** to generate the project:



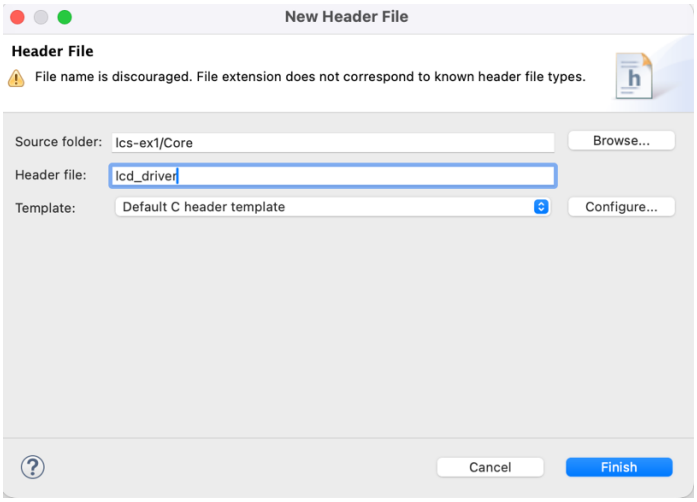
1.3.2. Creating Header and Source Files:

After the project has been generated by STM32CubeMX within STM32CubeIDE, we shall add new source file and header file.

Right click on Src folder and add new source file as following:



In similar manner, right click on Inc folder and add new header file with name of lcd_driver.h.



1.3.1.1 Developing the Header File:

```
/*
 * lcd_driver.h
 *
 *  Created on: Oct 4, 2025
```

```
 *      Author: tarekelfouly
 */

#ifndef INC_LCD_DRIVER_H_
#define INC_LCD_DRIVER_H_

#include "main.h"
void lcd_init (void);
void setCursor(int a, int b);
void lcd_send_string (char *str);
void lcd_clear (void);
void create_custom_char (uint8_t loc, char * data);
void display_custom_char (uint8_t loc);

#endif /* INC_LCD_DRIVER_H_ */
```

1.3.1.2 Developing the Source File:

```
/*
 * lcd_driver.c
 *
 *  Created on: Oct 4, 2025
 *      Author: tarekelfouly
 */

#include "lcd_driver.h"


static void LCD_WriteNibble(uint8_t nibble)
{
    // Set D4-D7 according to the nibble value
    HAL_GPIO_WritePin(D4_GPIO_Port, D4_Pin, (nibble & 0x01));
    HAL_GPIO_WritePin(D5_GPIO_Port, D5_Pin, (nibble & 0x02) >> 1);
    HAL_GPIO_WritePin(D6_GPIO_Port, D6_Pin, (nibble & 0x04) >> 2);
    HAL_GPIO_WritePin(D7_GPIO_Port, D7_Pin, (nibble & 0x08) >> 3);

    // Toggle EN pin to latch the nibble
    HAL_GPIO_WritePin(EN_GPIO_Port, EN_Pin, GPIO_PIN_SET);
    //HAL_Delay(1);  // Small delay to ensure the LCD latches the data
    HAL_GPIO_WritePin(EN_GPIO_Port, EN_Pin, GPIO_PIN_RESET);
    HAL_Delay(1);  // Small delay to ensure the LCD processes the nibble
}

static void LCD_WriteCommand(uint8_t command)
{
    // Set RS to 0 for command mode
    HAL_GPIO_WritePin(RS_GPIO_Port, RS_Pin, GPIO_PIN_RESET);

    // Send the higher nibble
    LCD_WriteNibble(command >> 4);
    // Send the lower nibble
    LCD_WriteNibble(command & 0x0F);
```

```
    HAL_Delay(1);  // Delay for command execution
}

static void LCD_WriteData(uint8_t data)
{
    // Set RS to 1 for data mode
    HAL_GPIO_WritePin(RS_GPIO_Port, RS_Pin, GPIO_PIN_SET);

    // Send the higher nibble
    LCD_WriteNibble(data >> 4);
    // Send the lower nibble
    LCD_WriteNibble(data & 0x0F);

    HAL_Delay(1);  // Delay for data processing
}


void setCursor(int a, int b)
{
      int i=0;
      switch(b){
      case 0:LCD_WriteCommand(0x80);break;
      case 1:LCD_WriteCommand(0xC0);break;
      case 2:LCD_WriteCommand(0x94);break;
      case 3:LCD_WriteCommand(0xd4);break;}
      for(i=0;i<a;i++)
            LCD_WriteCommand(0x14);
}


void lcd_send_string (char *str)
{
      while (*str) LCD_WriteData (*str++);
}


void lcd_init (void)
{
      // 4 bit initialisation
      HAL_Delay(50);  // wait for >40ms
      LCD_WriteCommand (0x3);
      HAL_Delay(5);  // wait for >4.1ms
      LCD_WriteCommand (0x3);
      HAL_Delay(1);  // wait for >100us
      LCD_WriteCommand (0x3);
      HAL_Delay(10);
      LCD_WriteCommand (0x2);  // 4bit mode
      HAL_Delay(10);

  // dislay initialisation
      LCD_WriteCommand (0x28); // Function set --> DL=0 (4 bit mode), N = 1
(2 line display) F = 0 (5x8 characters)
      HAL_Delay(1);
      LCD_WriteCommand (0x08); //Display on/off control --> D=0,C=0, B=0  ---
> display off
```

```
	HAL_Delay(1);
	LCD_WriteCommand (0x01);  // clear display
	HAL_Delay(1);
	HAL_Delay(1);
	LCD_WriteCommand (0x06); //Entry mode set --> I/D = 1 (increment
cursor) & S = 0 (no shift)
	HAL_Delay(1);
	LCD_WriteCommand (0x0C); //Display on/off control --> D = 1, C and B =
0. (Cursor and blink, last two bits)

}

void lcd_clear (void)
{

	#define LCD_CLEARDISPLAY 0x01
	LCD_WriteCommand(LCD_CLEARDISPLAY);
	HAL_Delay(100);

}

void create_custom_char (uint8_t loc, char * data)
{
	switch (loc)
	{
		case 0:LCD_WriteCommand(0x40);break;
		case 1:LCD_WriteCommand(0x40+8);break;
		case 2:LCD_WriteCommand(0x40+16);break;
		case 3:LCD_WriteCommand(0x40+24);break;
		case 4:LCD_WriteCommand(0x40+32);break;
		case 5:LCD_WriteCommand(0x40+40);break;
		case 6:LCD_WriteCommand(0x40+48);break;
		case 7:LCD_WriteCommand(0x40+56);break;
		default : break;
	}

	uint8_t cc_data[8];
	for (int i=0;i<8;i++)
	{
		cc_data[i]=*data++;
	}
	for (int i=0;i<8;i++)
	{
		LCD_WriteData(cc_data[i]);
	}

}

void display_custom_char (uint8_t loc)
{
	switch (loc)
			{
		case 0:LCD_WriteData(0);break;
		case 1:LCD_WriteData(1);break;
		case 2:LCD_WriteData(2);break;
```

```
            case 3:LCD_WriteData(3);break;
            case 4:LCD_WriteData(4);break;
            case 5:LCD_WriteData(5);break;
            case 6:LCD_WriteData(6);break;
            case 7:LCD_WriteData(7);break;
            default: break;
      }
}
```

2.3.1.3 Developing the main File:

Add these line

```
#include "lcd_driver.h"
#include <stdio.h>

char cc1[8] = {0x00, 0x00, 0x0A, 0x00, 0x11, 0x0E, 0x00, 0x00};  // smiley
char cc2[8] = {0x0E, 0x0E, 0x04, 0x0E, 0x15, 0x04, 0x0A, 0x0A};  // Robo
char cc3[8] = {0x08, 0x0C, 0x0E, 0x0F, 0x0E, 0x0C, 0x08, 0x00};  // arrow
char cc4[8] = {0x00, 0x04, 0x0E, 0x0E, 0x0E, 0x1F, 0x04, 0x00};  // bell
char cc5[8] = {0x00, 0x00, 0x0A, 0x15, 0x11, 0x0E, 0x04, 0x00};  // Heart
char cc6[8] = {0x00, 0x0E, 0x11, 0x11, 0x11, 0x0A, 0x1B, 0x00};  // omega
char cc7[8] = {0x0E, 0x10, 0x17, 0x12, 0x12, 0x12, 0x10, 0x0E};  // CT
char cc8[8] = {0x04, 0x04, 0x1F, 0x04, 0x04, 0x00, 0x1F, 0x00};  // +-
/* USER CODE END PV */
```

In user code begin 2 in main function:

```
lcd_init();

create_custom_char(0,cc1);
create_custom_char(1,cc2);
create_custom_char(2,cc3);
create_custom_char(3,cc4);
create_custom_char(5,cc6);
create_custom_char(6,cc7);
create_custom_char(7,cc8);
```

In user code begin 3 in while 1 loop:

```
setCursor(0, 0);
lcd_send_string("Custom characters");
HAL_Delay(200);
setCursor(0, 1);
for (int i=0; i<8; i++) {display_custom_char(i); HAL_Delay(300);}
HAL_Delay(700);
lcd_clear();
HAL_Delay(200);
```

1.3.2 Compile and flash

- Click the **Build** button

- Click the **Debug** button to run it step by step

1.3.3 Build and Display output on LCD

- Put jumper on PA1 - PA5 pins to connect DAC output to ADC input. The result is as follows

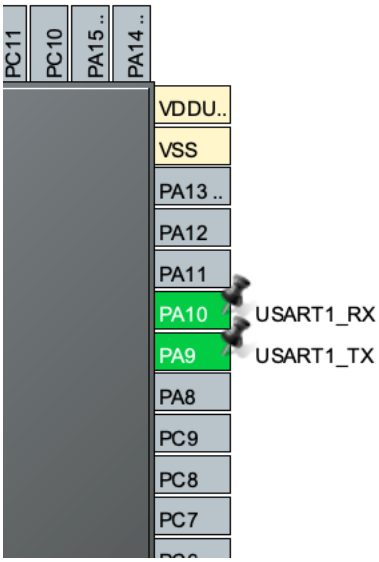
Part 2:

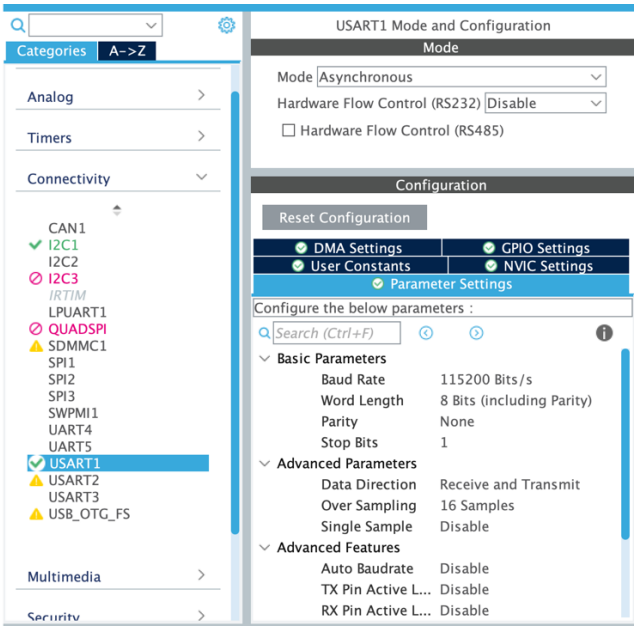
Objectives

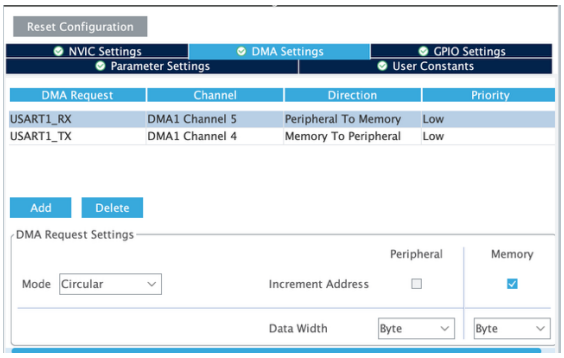
- Test the UART1 interface with DMA
- Connect the TX of UART1 to RX of UART1.
- Display the received values on the LCD.

2.1.1 Create project

2.2.2 Configure the additional Pins as follows







Add the following code to main

```
/* USER CODE BEGIN PFP */
uint8_t tx_buff[]={0,1,2,3,4,5,6,7,8,9};
uint8_t rx_buff[10];

/* USER CODE BEGIN 1 */
      char str[20];
  /* USER CODE END 1 */

/* USER CODE BEGIN 2 */

  HAL_UART_Transmit_DMA(&huart1, tx_buff, 10);
  HAL_UART_Receive_DMA(&huart1, rx_buff, 10);
while (1)
  {
    /* USER CODE BEGIN 3 */

            lcd_send_string("Custom characters");
            HAL_Delay(200);
            setCursor(0, 0);
            HAL_Delay (250);  // wait for a while
            for (int i=0; i<10; i++)
            {
                  sprintf(str, "%d", rx_buff[i]); lcd_send_string(str);
                  HAL_Delay(300);
            }
            HAL_Delay(700);
            lcd_clear();
            HAL_Delay(200);
  }
```

Then Display the output on the LCD.

Part 3:

Objectives

- Test the UART1 interface with DMA
- Transmit the received value from the Temperature and humidity sensor over the TX of UART1
- Connect the TX of UART1 to RX of UART1.
- Display the received values on the LCD.

Deliverables

- Report
- Video
- Project compressed file with the code.